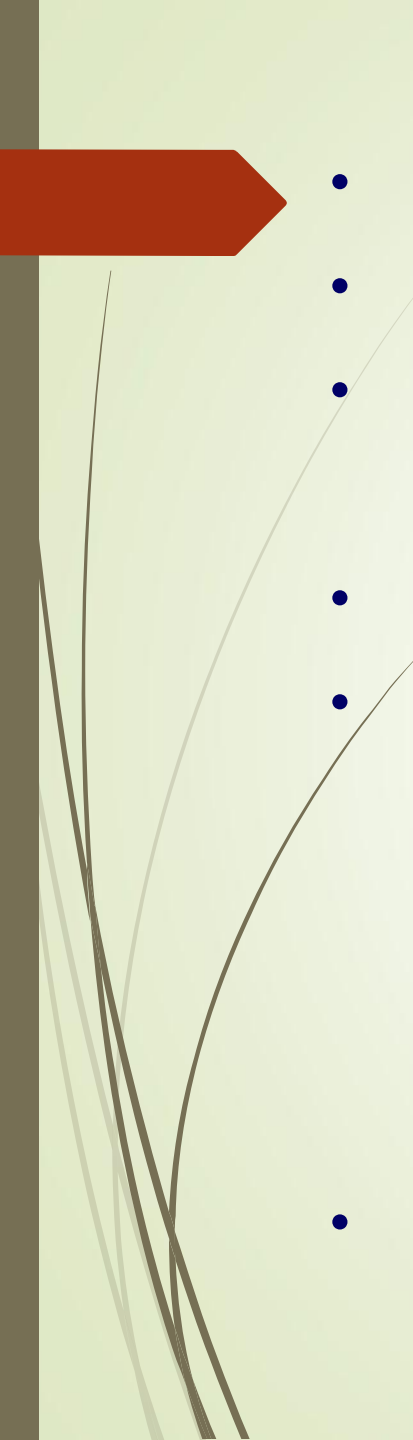




Développement d'applications multi-tiers

- 
- **Concept client-serveur**
 - **Architecture 2-tiers**
 - **Architecture 3-tiers**
 - **Exemple d'application avec Laravel**
 - **Introduction aux architectures N-tiers**
 - **Java EE**
 - **Définition & objectifs**
 - **Architecture Java EE**
 - **Architecture d'une application Java EE**
 - **Exemple d'application avec SpringBoot**
 - **Serveur d'application**



Concept client-serveur

- L'architecture client/serveur désigne un mode de communication entre plusieurs ordinateurs d'un réseau qui distingue un ou plusieurs postes clients du serveur : chaque logiciel client peut envoyer des requêtes à un serveur.
- Un serveur peut être spécialisé en serveur d'applications, de fichiers, de terminaux, ou encore de messagerie électronique.

Concept client-serveur

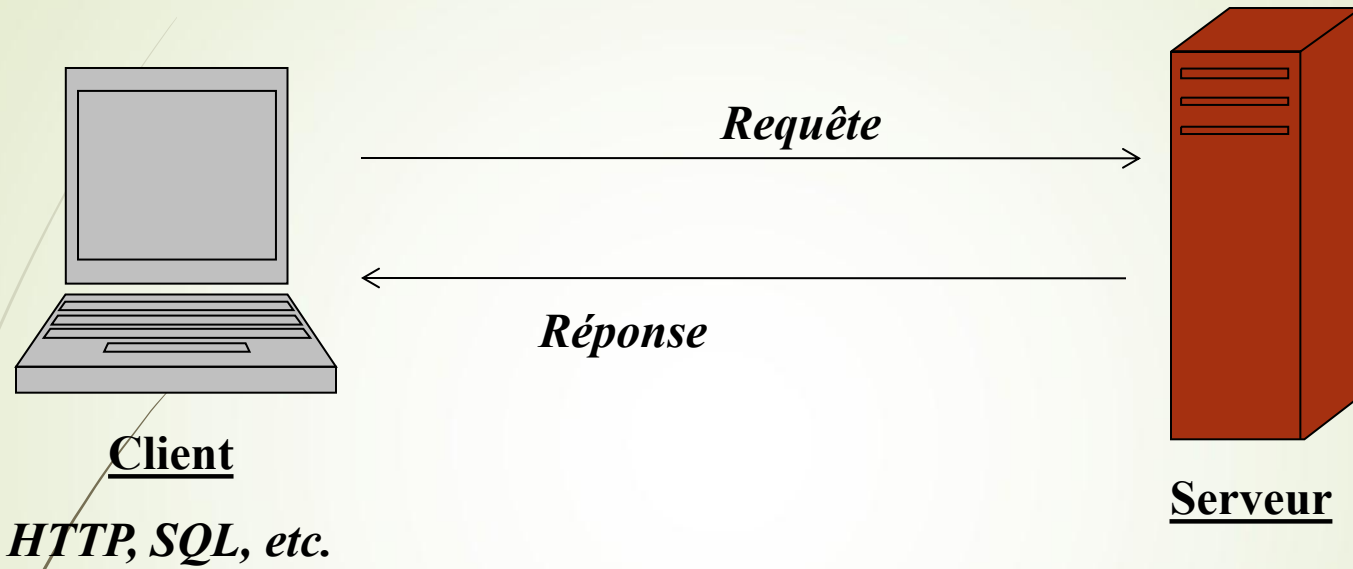
- Caractéristiques d'un serveur :
 - il est passif (ou maître) ;
 - il est à l'écoute, prêt à répondre aux requêtes envoyées par des clients ;
 - dès qu'une requête lui parvient, il la traite et envoie une réponse.

Concept client-serveur

- Caractéristiques d'un client :
 - il est actif (ou esclave) ;
 - il envoie des requêtes au serveur ;
 - il attend et reçoit les réponses du serveur.
- Le client et le serveur doivent bien sûr utiliser le même protocole de communication.
- Un serveur est généralement capable de servir plusieurs clients simultanément.

Architecture 2-tiers

- L'architecture à deux niveaux (aussi appelée architecture 2-tier, tier signifiant étage en anglais) caractérise les systèmes clients/serveurs dans lesquels le client demande une ressource et le serveur la lui fournit directement.
- Cela signifie que le serveur ne fait pas appel à une autre application afin de fournir le service.



Architecture client-serveur classique : 2-tier

Architecture 2-tiers

- Client / serveur de base, avec 2 éléments
 - Client : présentation, interface utilisateur
 - Serveur : partie persistance, gestion physique des données

Architecture 2-tiers

- Les services métier / la partie applicative peuvent être:
- Soit entièrement coté client, intégrés avec la présentation
 - La partie serveur ne gère que les données
 - Ex : traitement de texte avec serveur de fichiers distants
 - Ex : application accédant à une BDD distante

Architecture 2-tiers

- Soit entièrement coté serveur
 - La partie client ne gère que l'interface utilisateur
 - L'interface utilisateur peut même être exécutée sur le serveur
 - Fonctionnement mode terminal
 - L'utilisateur a simplement devant lui écran / clavier / souris pour interagir à distance avec l'application s'exécutant entièrement sur le serveur
- Soit découpés entre la partie serveur et la partie client

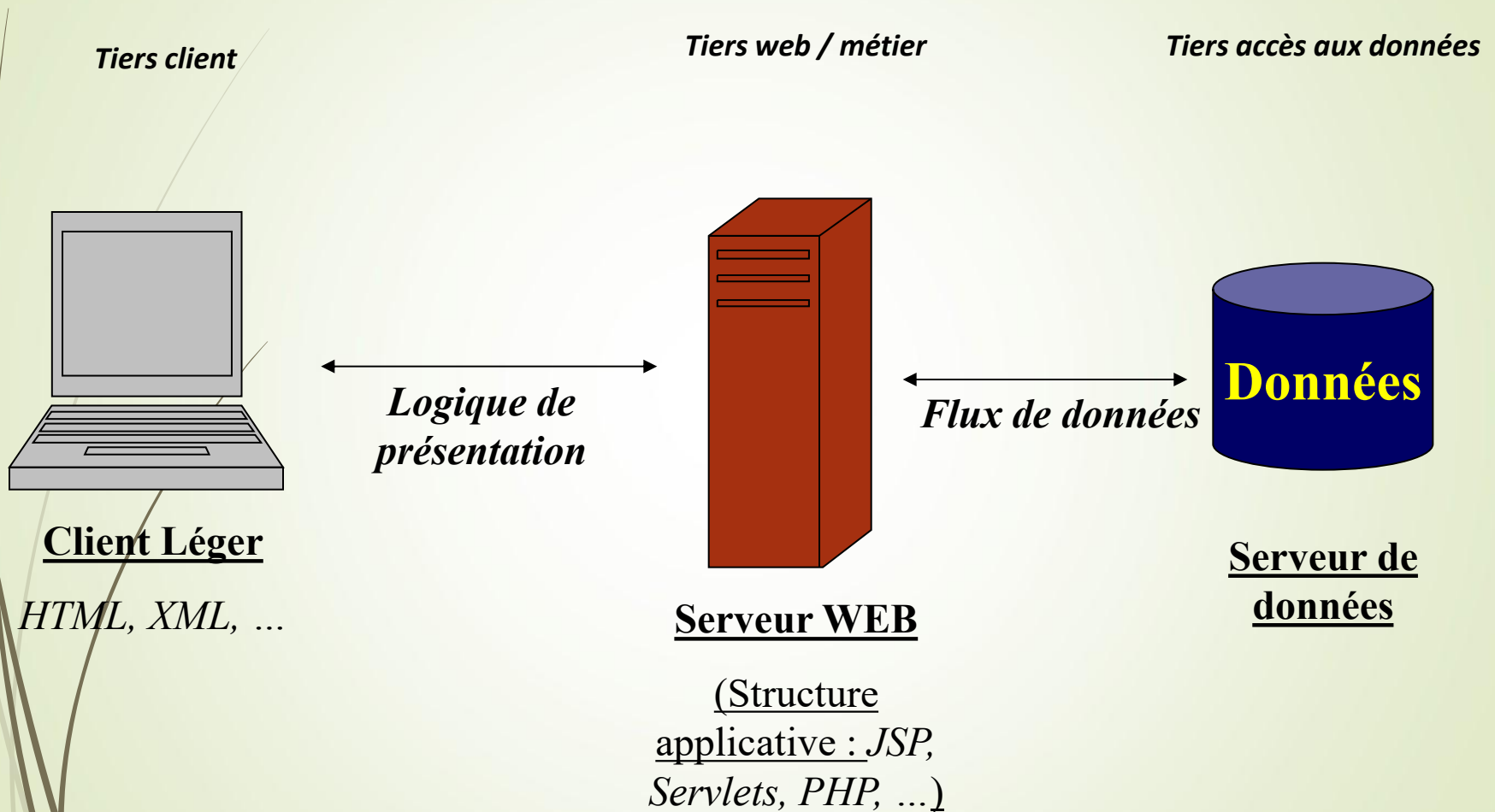


Architecture 3-tiers



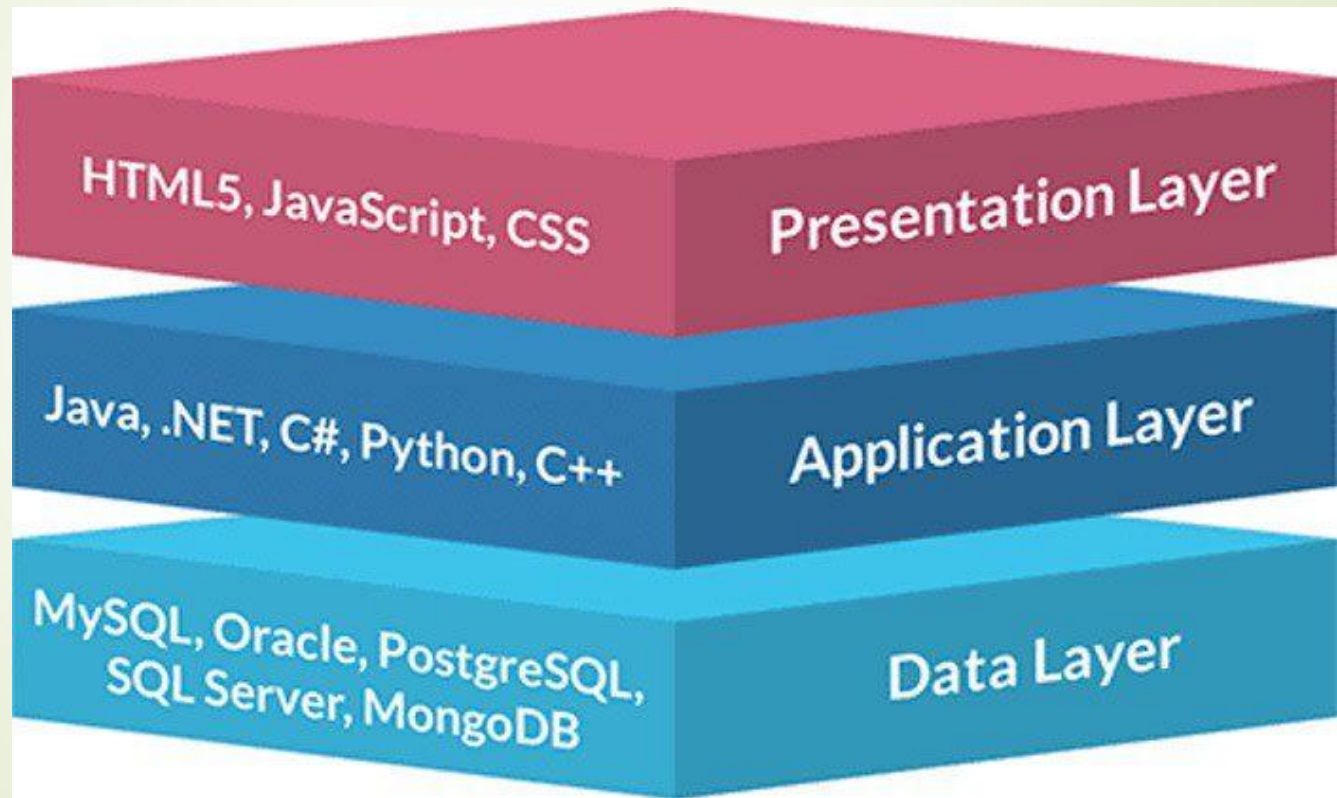
Architecture 3-tiers

- L'architecture 3-tiers est un pattern d'architecture de code source qui permet de séparer les différentes couches de l'application.
- À l'instar du pattern MVC, l'architecture 3 tiers propose de séparer la couche donnée, la couche métier et la couche interface utilisateur.



Architecture client-serveur web : 3-tiers

Architecture 3-tiers



Architecture 3-tiers

- **Tiers client (Présentation): client léger et *universel***
 - Vision du SI pour les utilisateurs finaux
 - Application utilisable même dans un parc hétérogène
- **Tiers Web (Application / Métier) : serveur web (+ *serveur d'applications*)**
 - Implantation de la logique applicative (*traitement des requêtes émises par les clients*)
 - Logique de gestion et access aux données stockées dans la base de données
- **Tiers resource (Persistance) : serveur de données**
 - Dédié au stockage et à l'accès aux données (*BD et application existantes*)

Architecture 3-tiers

➤ La couche présentation.

- Navigateur web sur machine cliente
- Client léger
- Service web (API)
- Affichage de contenu HTML

Architecture 3-tiers

➤ La couche persistance

- Serveur(s) de BDD de données
- Accès aux données

Architecture 3-tiers

- **la couche traitement (applicative / métier)**
 - mise en œuvre de l'ensemble des règles de gestion et de la logique applicative
 - métier des données
 - découpler la logique métier des opérations de base de données
 - **Avantage** : faire évoluer le métier et la base de données séparément
 - **Inconvénient** : une seule brique représente la couche applicative et métier

Architecture 3-tiers

➤ Avantages

- Séparation entre le client, l'application et le stockage

➤ Limites

- Manque de séparation au niveau de l'application
- Application : logique métier, logique fonctionnelle, accès aux données.

➤ Solution

- scinder la couche applicative en plusieurs niveaux
- architecture multi-tiers



Architectures multi-tiers

Architecture multi-tiers

Définition

- L'architecture multi-tier ou encore appelée N-tier, est une architecture client-serveur dans laquelle une application est exécutée par plusieurs composants logiciels distincts.
- Cette séparation facilite la gestion de chacun séparément car le fait de travailler sur l'un n'affecte pas les autres, isolant ainsi les problèmes qui pourraient survenir.

Architectures multi-tiers

Avantages

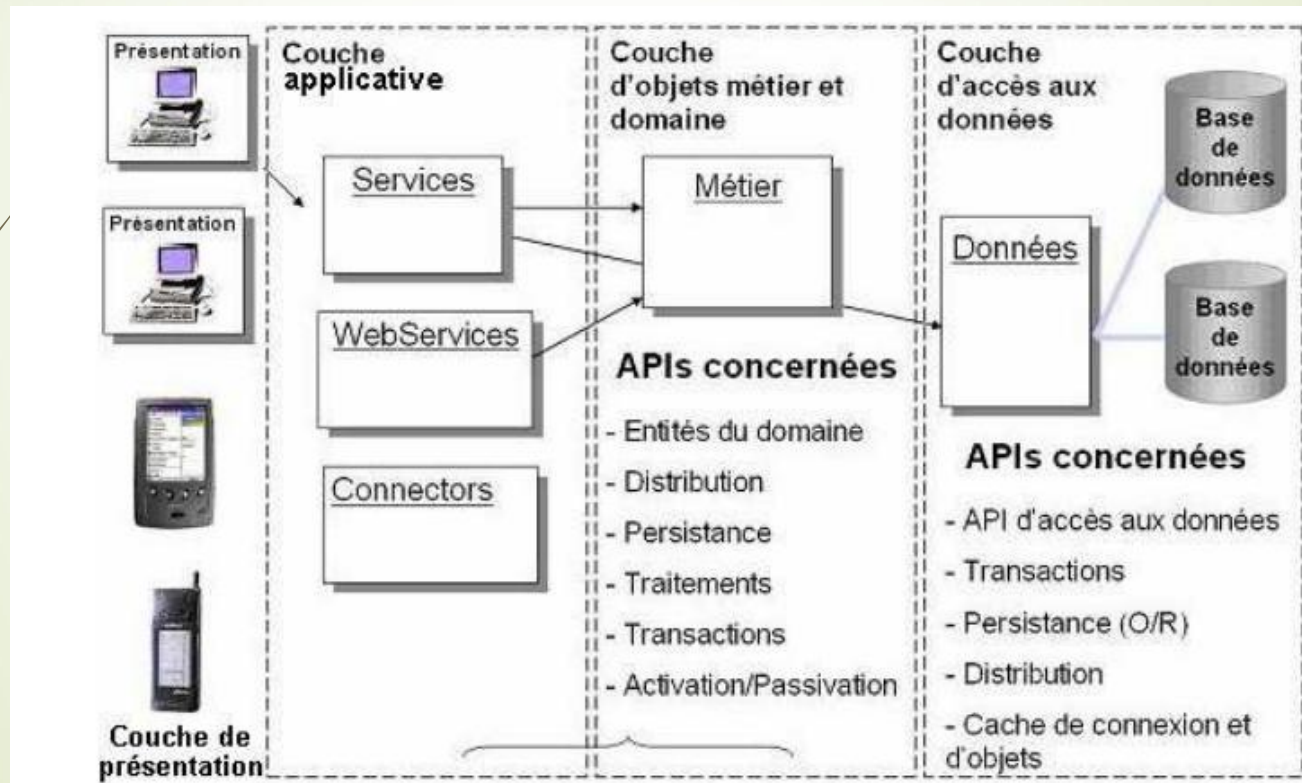
- Généralisation du modèle 3-tiers.
- Prend en compte l'évolutivité du système.
- Existence d'un **tiers métier** :
 - Regroupe la logique métier de l'entreprise.
 - Utilisé par différentes applications clientes.
 - Séparation en couches au niveau de l'architecture logicielle.
 - Exemple : *présentation, application, métier, données.*

Architecture multi-tiers

Avantages

- Le lien entre les niveaux est défini et limité à des interfaces
- Les interfaces assurent la modularité et l'indépendance technologique et topologique de chaque niveau
- Réutilisation de services existants
- Découplage des aspects métiers et technique et des services entre eux : meilleure modularité
- Facilite évolution : nouvelle version de service
- couplage faible entre les services

Architecture multi-tiers



Architecture multi-tiers

Les différentes couches d'une architecture 4-tier :

- **La couche de présentation** contient les différents types de clients, léger (HTML, XML, JavaScript, API, ...) ou lourd (Applet)
- **La couche applicative** contient les traitements représentant les règles métier (créer un compte de facturation, calculer un amortissement ...)
- **La couche métier** est représentée par les objets du domaine, c'est à dire l'ensemble des entités persistantes de l'application (Facture, Client ...)
- **La couche d'accès aux données** contient les usines d'objets métier, c'est à dire les classes chargées de créer des objets métier de manière totalement transparente, indépendamment de leur mode de stockage (SGBDR, Objet, Fichiers, ...)

Architecture multi-tiers

Couche présentation

- L'interaction avec l'utilisateur
- La logique de présentation
- Le traitement des données retournées par les services métiers et leur présentation à destination de l'utilisateur

Architecture multi-tiers

Couche applicative

- ▀ la logique applicative, les traitements, décrivant les travaux à réaliser par l'application.
- ▀ les traitements, regroupant les contrôles effectués au niveau du dialogue avec la couche présentation, visant essentiellement le contrôle et l'aide à la saisie.

Architecture multi-tiers

Couche métier

- Un objet métier est un concept ou une abstraction ayant un sens pour des acteurs (partie prenante interne d'une organisation (entreprise))
- L'objet métier permet de décrire les entités manipulées par les acteurs dans le cadre de la description du métier
- Exemple
 - Mon métier consiste à gérer les inscriptions des étudiants
 - Les objets sont : l'étudiant, la formation, le paiement ...

Architecture multi-tiers

Couche métier

- Les objets métiers sont représentés par l'ensemble des objets persistants du domaine de l'application
 - Une facture, un bon de commande, ou tout autre objet nécessitant d'être stocké en base
- Il existe deux méthodes pour accéder aux données;
 - La première consiste à accéder directement aux sources de données
 - La deuxième consiste à s'appuyer sur des objets métiers (client, fournisseur, facture ...) afin de masquer la complexité d'accès aux données.
- La couche objet métier assure l'indépendance totale entre le client et le type de stockage utilisé (SGBDR, Fichiers XML, ...)

Architecture multi-tiers

Couche métier

► Exemple:

- Un objet AssuréSocial possèdera par exemple une méthode débit() et une méthode crédit() qui à chaque appel iront modifier les données dans une
 - base de données
 - Ou des Fichiers

Architecture multi-tiers

Couche métier

- Pour gérer ces objets métiers, un environnement d'exploitation est nécessaire : les serveurs d'objets
- Fournit les services essentiels suivants
 - Gestion du cycle de vie des objets :
 - Fonctionnalités de base permettant la création, la recherche, la manipulation, et la destruction des objets.
 - Gestion de la persistance :
 - Synchronisation de l'état des objets sur un support de persistance afin d'assurer la sauvegarde durable de l'état des objets.
 - Gestion des transactions:
 - Permet d'assurer l'intégrité des données et de gérer la concurrence d'accès

Architecture multi-tiers

Couche métier

- Gestion de la montée en charge :
 - Mécanisme (pools d'objet, cache transactionnel, ect ...) pour améliorer les performances des applications accédant de manière concurrente aux objets.
- Gestion de la sécurité :
 - Mécanisme d'authentification et de contrôle d'accès aux objets.
 - Définition de permissions sur des opérations de lecture, de mise à jour, et sur les appels aux traitements métier permettant de définir des restrictions d'accès basées sur la définition de groupes d'utilisateurs et de rôles.

Architecture multi-tiers

Couche métier

- Exemple serveurs d'objets :
 - **EJB(Entreprise Java Beans) de JEE**

Architecture multi-tiers

Couche données

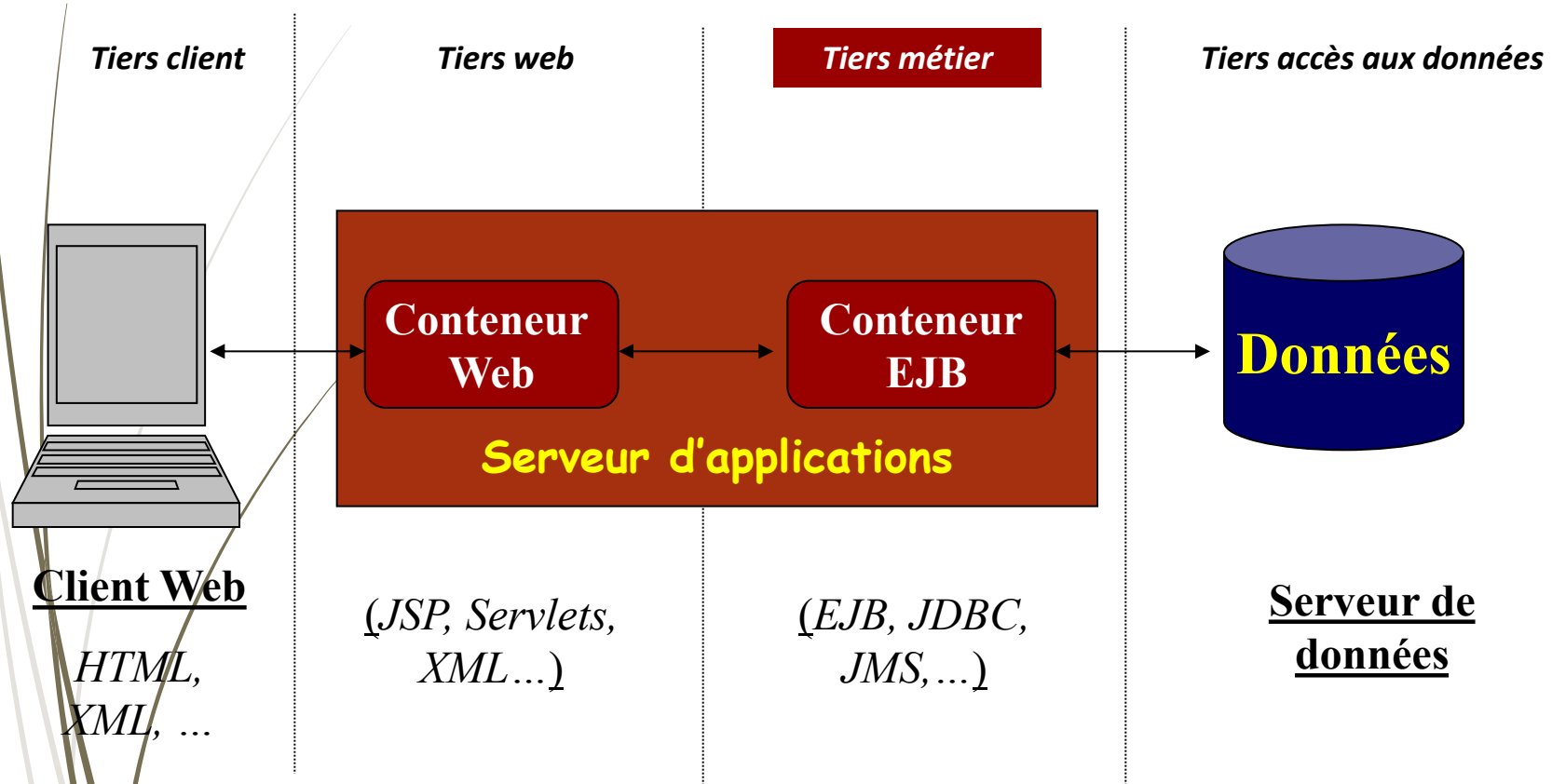
- ▀ Accès aux données
- ▀ Serveur(s) de BDD de données (stockage)

Architecture multi-tiers

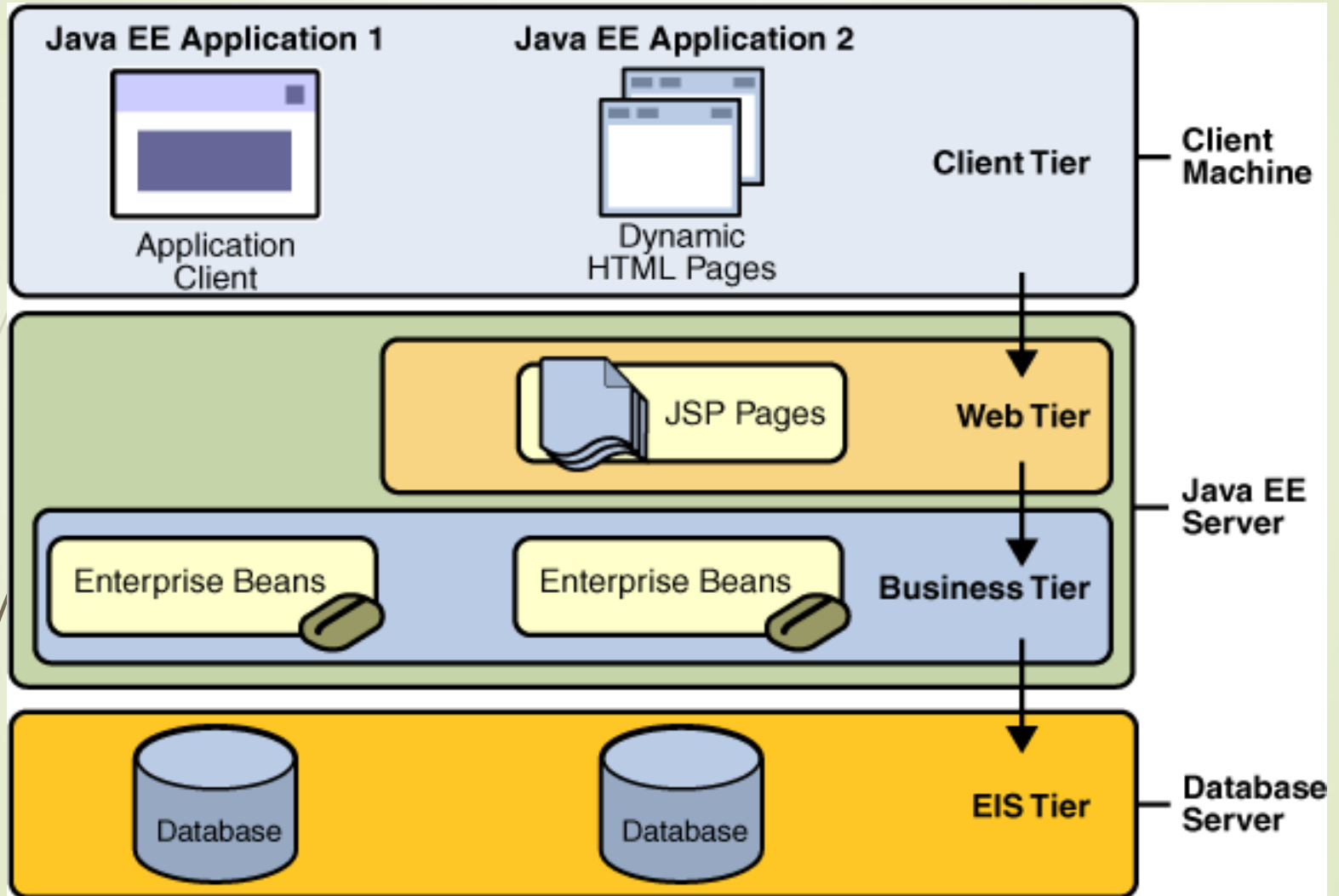
Valeur ajoutée des architectures multi-tiers

- Cette séparation par couches de responsabilités sert à découpler au maximum une couche de l'autre afin d'éviter l'impact d'évolutions futures de l'application.
- Par exemple : si l'on est amené à devoir changer de base de données relationnelle, seule la couche d'accès aux données sera impactée, la couche de service et la couche de présentation ne seront pas concernées car elles auront été découplées des autres.

Architectures multi-tiers: cas de Java



Architectures d'applis multi-tiers : Java EE



Java EE

La solution Multi-tier de Java pour le web dynamique et les applications d'entreprise en réponse aux limites des architectures C/S

Java EE

Définition

- Java EE est un ensemble de spécifications pour définir une architecture multi-niveaux permettant de développer des systèmes d'information d'entreprise (EIS) afin de répondre aux besoins de divers industries.
- Ces spécifications sont définies pour répondre aux demandes des entreprises (industries, banques, compagnies financières, tech ...) pour développer des applications autour de leur SI et de leur métier.
- Fournit une API et un environnement d'exécution pour développer et exécuter des logiciels d'entreprise, y compris des services réseau et Web, et d'autres applications multi-tiers à grande échelle, évolutives, fiables et sécurisées
- La plate-forme étend Java Platform Standard Edition (Java SE) en fournissant une API de mapping objet-relationnel, des architectures distribuées et multi-tiers, et des services web

Java EE

Objectifs

- ✓ Réduire le cout et complexité de développement
- ✓ Faciliter le développement de nouvelles applications à base de composants
- ✓ Permettre aux applications Java EE d'être rapidement déployées et facilement améliorées
- ✓ Intégration avec les systèmes d'information existants
- ✓ Support pour les applications « critiques » de l'entreprise
 - Disponibilité, tolérance aux pannes, montée en charge, sécurité...

Java EE

C'est un standard du marché car il offre :

- ✓ une simplification de l'architecture, du développement et de la maintenance
- ✓ un support du transactionnel et de la scalabilité
- ✓ une intégration homogène avec les SI existants
- ✓ une indépendance sur le choix des serveurs, des outils et des composants

J2E s'appuie sur des concepts objet :

- Classe : type d'objet caractérisé par sa structure de données (attributs) et son comportement (méthodes)
- Objet : instance de classe.
- Héritage : Mécanisme permettant à une classe d'objets de bénéficier de la structure de données et du comportement d'une classe "mère", tout en lui permettant de les affiner et ce, afin de prendre en compte les spécificités de la classe "fille", sans avoir cependant à redéfinir ce que les deux classes ont de commun.
- Abstraction : Mécanisme permettant la dissociation entre la déclaration d'une classe et son implémentation.
- Polymorphisme : Mécanisme permettant d'associer à un comportement, une implémentation différente en fonction de l'objet auquel on se réfère.
- Encapsulation : Mécanisme permettant de dissimuler les détails du fonctionnement interne d'une classe aux autres classes.

Java EE

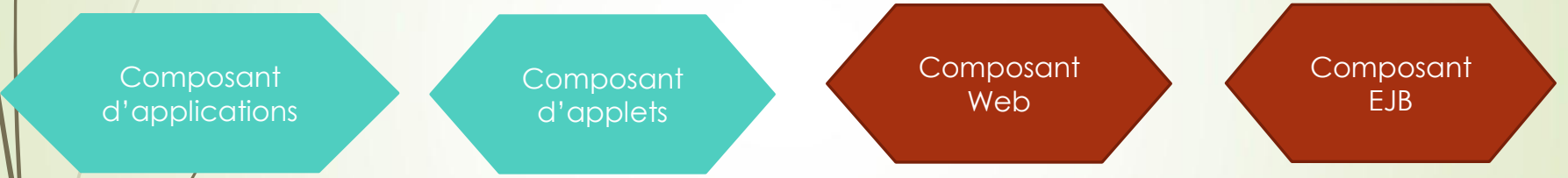
J2EE est une architecture de composants :

- Objectif des composants :
 - avoir des briques de bases réutilisables.
- Définition d'un composant :
 - module logiciel,
 - exporte différents attributs, propriétés et méthodes,
 - est prévu pour être configuré,
 - est prévu pour être installé,
 - fournit un mécanisme lui permettant de s'auto-décrire.

Architecture Java EE

Architecture orientée composant

- La spécification Java EE définit les composants Java EE suivants :



Composant d'applications

Composant d'applets

Composant Web

Composant EJB

- Les clients d'application et les applets sont des composants qui s'exécutent sur le client.
- Les composants technologiques Java Servlet, JavaServer Faces (JSF) et JavaServer Pages (JSP) sont des composants Web qui s'exécutent sur le serveur.
- Les composants EJB (Entreprise Beans) sont des composants métier qui s'exécutent sur le serveur.

Architecture Java EE

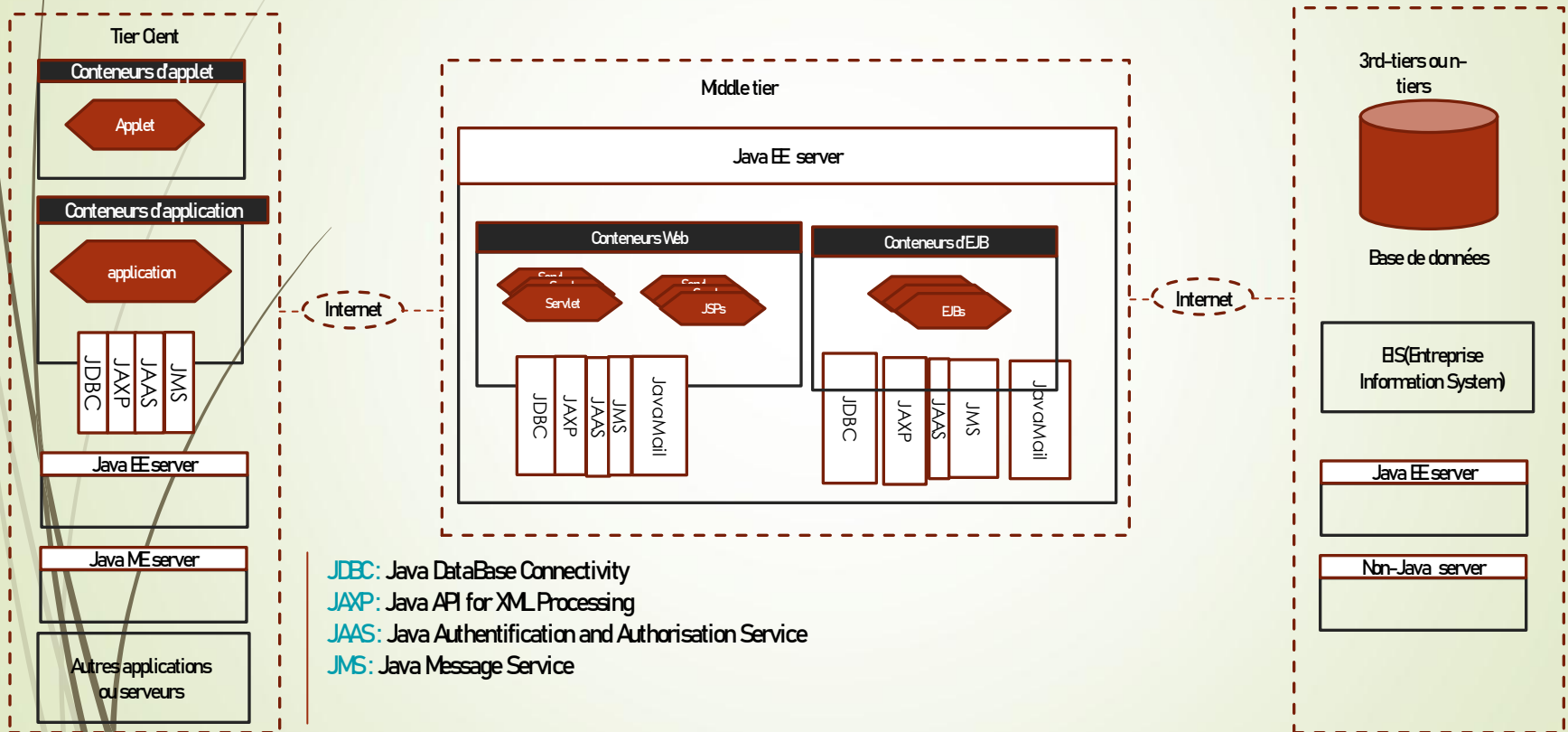
Architecture orientée composant

Le développement d'applications Web repose sur trois composants J2EE principaux :

- Les servlets : ce sont des programmes Java exécutés sur un serveur (par sa JVM). Ils permettent d'étendre le comportement du serveur dynamiquement.
- Les JSP : ce sont des pages HTML incluant du code JAVA (stocké à l'intérieur de balises).
- Les EJB : ce sont des entités de traitement s'exécutant dans un environnement adapté (conteneur) et possédant des mécanismes de configuration et d'installation.

Architecture Java EE

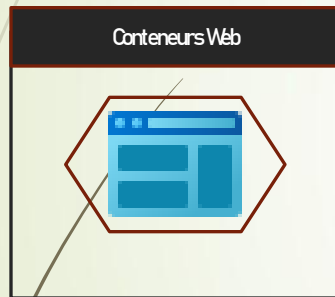
Architecture orientée composant : la réponse n-tiers de Java aux limites de l'architecture C/S



Architecture Java EE

Tiers client

- Les types de clients Java EE:



- Web



- Application

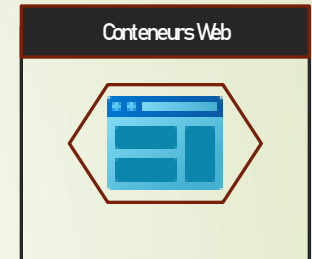


- Applet

Architecture Java EE

Tiers client

- Un client Web se compose de deux parties :
 - ✓ Pages Web dynamiques contenant divers types de langage de balisage (HTML, XML, etc.), générés par des composants Web exécutés dans le tiers Web
 - ✓ Un navigateur Web, qui interprète/exécute les pages reçues du serveur
- Un client Web est parfois appelé client léger.



Client Web

Architecture Java EE

Tiers client

■ Clients Application

- ✓ Un client d'application s'exécute sur une machine cliente. Il a généralement une interface utilisateur graphique, mais une interface en ligne de commande est aussi possible.
- ✓ Les clients d'application accèdent directement aux EJBs s'exécutant dans le tier métier.
- ✓ Cependant, si les exigences de l'application le justifient, un client d'application peut ouvrir une connexion HTTP pour établir une communication avec un servlet exécuté dans le niveau Web.

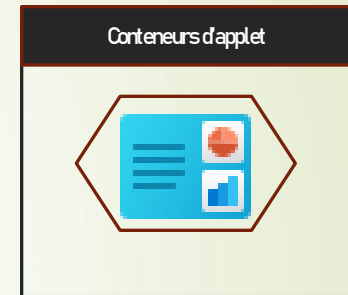


Client Application

Architecture Java EE

Tiers client

- Applets
 - ✓ Une page Web reçue du tiers Web peut inclure une applet intégrée. Écrit dans le langage de programmation Java.
 - ✓ une applet est une petite application cliente qui s'exécute dans la machine virtuelle Java installée dans le navigateur Web.



Client Applet

Architecture Java EE

Tiers application(web)

- Au niveau du tiers Web, le conteneur Web gère deux composants essentiels : Servlet et JSP
- Conteneur Web
 - ✓ Gère l'exécution des servlets et des JSP
 - ✓ Partie d'un serveur Web ou d'applications
 - ✓ Prend en charge le HTTP



Architecture Java EE

Tiers application(web)

Composant JSP



- JSP: Java Server Page, page HTML pouvant contenir du code java
- Solution Java aux templates *HTML*.
- Permettent de séparer la page statique des pages *HTML* de la partie générée dynamiquement avec du code Java.
- Le code Java est inclus au moyen des balises `<%` et `%>`.
- Les pages **JSP** ont une extension `.jsp`



Architecture Java EE

Tiers application(web)

Composant Servlet



Programmes Java qui s'exécutent dans un serveur Web selon un fonctionnement en mode **requête / réponse**.

Classes qui traitent dynamiquement les requêtes (i.e. requêtes http) et construisent les réponses (i.e. réponse http)

- ✓ Générer dynamiquement des pages **html** en réponse aux requêtes
- ✓ Peut également envoyer des données en réponse sous d'autres formes comme **XML** ou des **objets Java sérialisés**

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Bonjour extends HttpServlet {

    public void doGet(HttpServletRequest req,
        HttpServletResponse rep) throws IOException,
        ServletException
    {
        rep.setContentType( "text/html" );
        PrintWriter out = rep.getWriter();
        out.println( "<html>" );
        out.println( "    <head> <title>Bonjour
        </title> </head> " );
        out.println( "    <body>" );
        out.println( "        <h1>Bonjour</h1>" );
        out.println( "    </body>" );
        out.println( "</html>" );
    }
}
```

Architecture Java EE

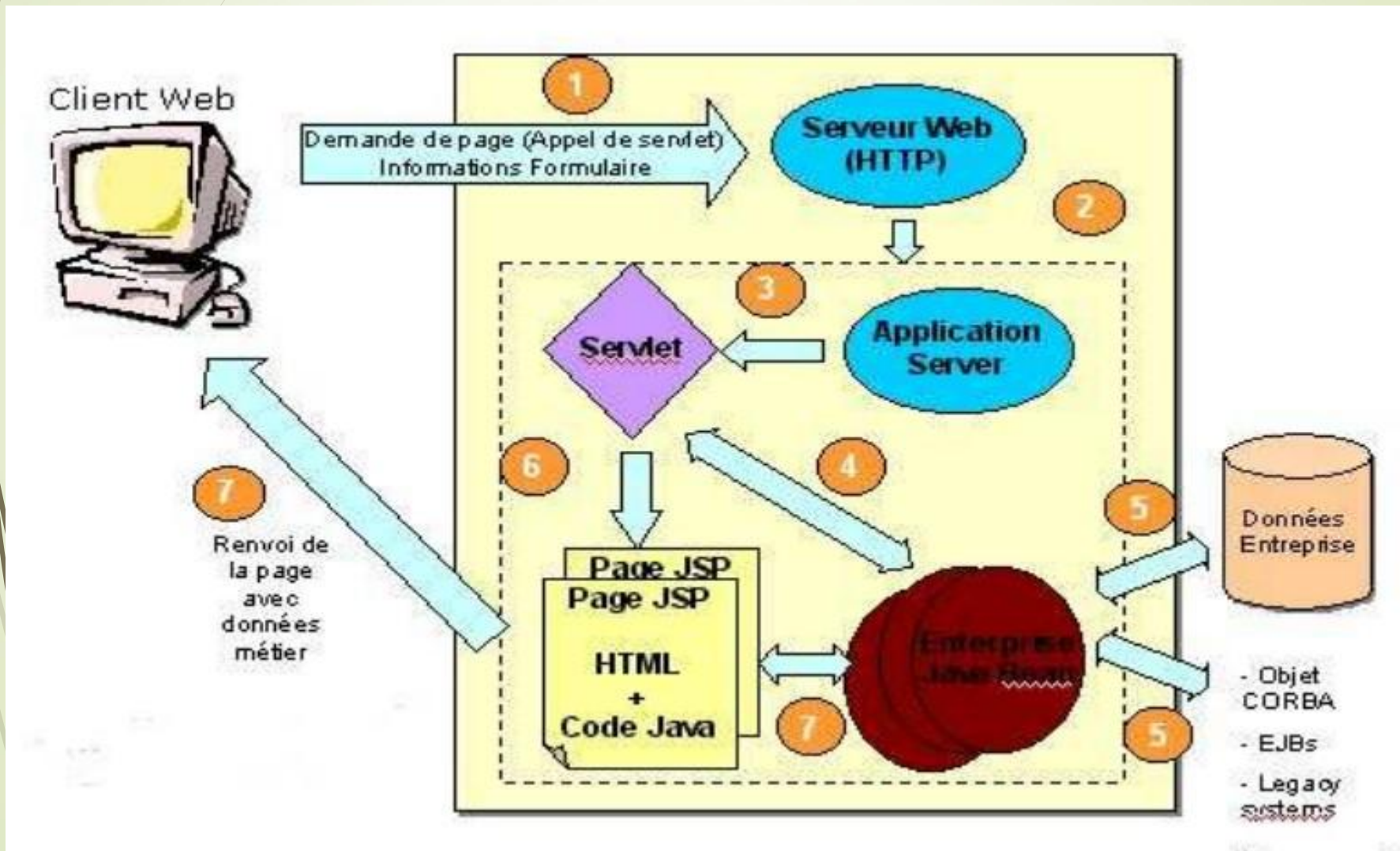
Tiers métier

Composant EJB

- EJB (Enterprise JavaBeans) est un composant serveur standard de Jakarta EE destiné à encapsuler la logique métier, avec des fonctionnalités transversales automatiques comme :
 - ❖ Transactions
 - ❖ Sécurité
 - ❖ Persistance (avec JPA)
 - ❖ ...

Architecture Java EE

Communication entre tiers



Architectures d'applis multi-tiers : Spring Boot

- ❑ **Framework basé sur le framework Spring**, qui simplifie le développement en Java en automatisant la configuration
- ❑ Utilise des technologies similaires à JEE, mais avec une **approche différente**
- ❑ Se lance **sans serveur d'application externe** (Tomcat est intégré par défaut)
- ❑ **Spring Boot** : une **solution pratique, moderne et populaire**, qui réinvente et simplifie les concepts de JEE.

Architectures d'applis multi-tiers : Spring Boot

- ❑ Exemple d'application multi-tiers avec **Spring Boot (Java)** pour le backend et **Angular** pour le frontend

Architectures d'applis multi-tiers : Spring Boot

- Spring Boot :

Dans `application.properties` :

```
properties
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo  
spring.datasource.username=root  
spring.datasource.password=admin  
spring.jpa.hibernate.ddl-auto=update
```

Architectures d'applis multi-tiers : Spring Boot

1.1. Entité Product.java

java

```
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private double price;

    // Getters and Setters
}
```

Architectures d'applis multi-tiers : Spring Boot

1.2. Repository `ProductRepository.java`

java

`@Repository`

```
public interface ProductRepository extends JpaRepository<Product, Long> {}
```

Architectures d'applis multi-tiers : Spring Boot

1.3. Service ProductService.java

java

```
@Service
public class ProductService {
    @Autowired
    private ProductRepository repository;

    public List<Product> findAll() {
        return repository.findAll();
    }

    public Product save(Product product) {
        return repository.save(product);
    }

    public void delete(Long id) {
        repository.deleteById(id);
    }
}
```



Architectures d'applis multi-tiers : Spring Boot

1.4. Contrôleur ProductController.java

```
java

@RestController
@RequestMapping("/api/products")
@CrossOrigin(origins = "*") // autoriser Angular
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping
    public List<Product> getAll() {
        return service.findAll();
    }

    @PostMapping
    public Product add(@RequestBody Product product) {
        return service.save(product);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

Architectures d'applis multi-tiers : Spring Boot

```
@Injectable({
    providedIn: 'root'
})
export class ProductService {
    private apiUrl = 'http://localhost:8080/api/products';

    constructor(private http: HttpClient) {}

    getProducts(): Observable<Product[]> {
        return this.http.get<Product[]>(this.apiUrl);
    }

    addProduct(product: Product): Observable<Product> {
        return this.http.post<Product>(this.apiUrl, product);
    }

    deleteProduct(id: number): Observable<void> {
        return this.http.delete<void>(`${this.apiUrl}/${id}`);
    }
}
```

Le serveur d'application

Wikipedia :

« Un **serveur d'application** est un serveur sur lequel sont installées les applications utilisées par les usagers. Ces applications sont chargées sur le serveur d'applications et accédées à distance, souvent par réseau. »

Le serveur d'application

Environnement complet de développement coté serveur permettant de :

- mettre en œuvre des **applications distribuées**,
fabriquées à base de **composants**
- et de les rendre accessibles à des clients.

Le serveur d'application

- ✓ Permet :
 - d'optimiser les accès aux données, les ressources partagées
 - de mettre en oeuvre des composants métier sophistiqués
- ✓ Accédé par les clients depuis HTTP ou d'autres protocoles
- ✓ Client des serveurs de données

Le serveur d'application

- ✓ Fournit de nombreuses fonctions :
 - support du multi-thread
 - optimisation des connexions aux BD
 - support optimisé des transactions
 - sécurité des informations
 - persistance des objets métier.
- ✓ Implémente plusieurs services utiles aux composants et au fonctionnement des applications.

Le serveur d'application

Principales fonctionnalités d'un serveur web

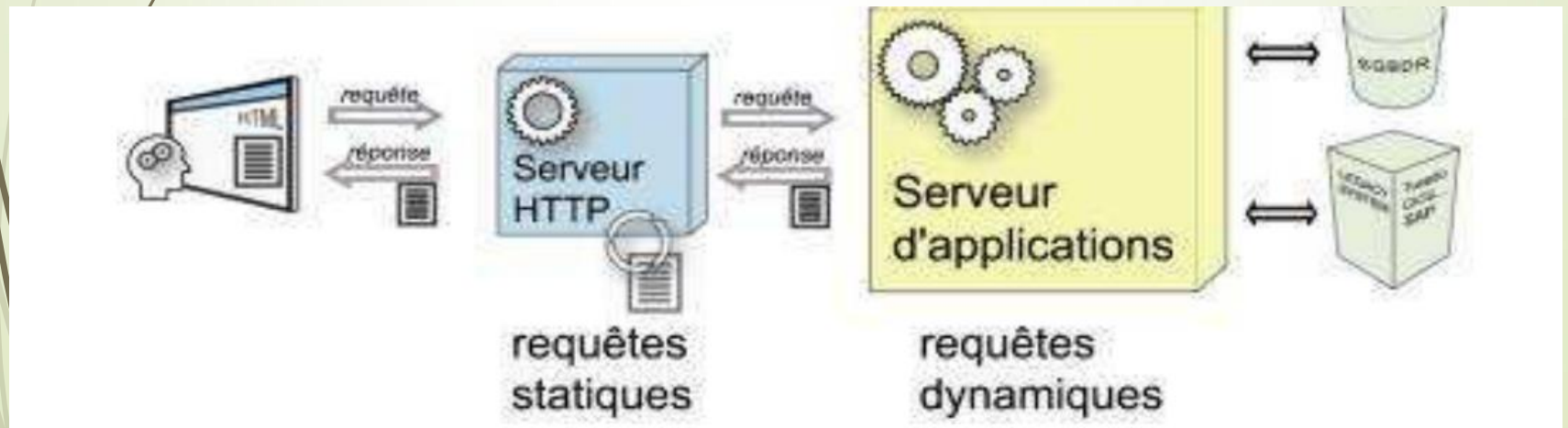
- ✓ Réceptionner la requête
- ✓ Re-router les requêtes dynamiques
- ✓ Rechercher les pages statiques
- ✓ Encapsuler les pages dans la réponse
- ✓ Émettre la réponse



Le serveur d'application

Principales fonctionnalités d'un serveur d'application

- ✓ Réceptionner la requête
- ✓ Construire la réponse dynamique
- ✓ Renvoyer la réponse au serveur Web





Les services d'un serveur d'application (1/2)

- ✓ Service de nommage (stockage des références sur des ressources)
- ✓ Service de gestion des transactions (Gestion des transactions locales et distribuées)
- ✓ Service de gestion de la disponibilité des applications
(Capacité à gérer un grand nombre d'accès et de requêtes simultanés.)



Les services d'un serveur d'application (2/2)

- ✓ Service de sécurité (Gestion utilisateurs)
- ✓ Service d'administration
- ✓ Déploiement et configuration des applications
- ✓ Service d'accès aux données
- ✓ Service de gestion de messages

Serveurs d'applications Java

- Apache Tomcat
- GlassFish Sun
- Jboss
- Jonas
- WebSphere IBM
- Oracle Application Server



Sources

- <https://openclassrooms.com/fr/courses/4503526-organisez-et-packagez-une-application-java-avec-apache-maven/4609181-decoupez-votre-projet-en-couches-applicatives>
- Cours Architecture MultiTiers du PR Moussa LO